



《软件测试》

第十三章 软件测试的 拓展与提高



授课人：高利鹏

日期：2025年12月24日

目 录

CONTENTS

01

企业测试实践

02

CMMI 和软件测试

03

DevOps

04

小结

学习目标

- ◆ 了解企业测试的主要过程
- ◆ 了解基于CMMI的软件测试流程
- ◆ 了解DevOps

目 录

CONTENTS

01

企业测试实践

02

CMMI 和软件测试

03

DevOps

04

小结

1.1 测试计划

测试文档:

主要包括:

测试计划: 提供产品测试工作的概述

测试说明: 主要对测试用例的信息进行说明

测试报告: 描述了已经完成的测试, 并对测试结果进行评价。

测试文档主要作用:

- (1) 测试计划文档为完成测试任务提供便利。
- (2) 测试计划文档改善测试任务与测试过程之间的联系。
- (3) 测试计划文档为组织、规划与管理测试项目提供支持。

1.1 测试计划

● 使用以下方式可以提高测试人员测试程序的能力

1. 提高测试覆盖率：

测试计划要有程序特征清单，为了创建该清单，需要对程序的特征进行寻找和统计。

2. 避免不必要的重复：

当对测试任务进行核查时，对已经测试和没有测试的内容进行查找和核对。

3. 分析程序并快速挑选好的测试用例

4. 提供最终测试的结构：

由于软件产品通常是要在比较短的时间周期内完成开发，一般留给最后测试的时间安排都很少，而软件测试计划可以帮助确保最重要的测试得到运行。

5. 检查完整性：

完整的测试计划不会忽略程序中的问题，例如，忽略了软件缺陷类型、测试类别和程序区域等。

1.1 测试计划

编写测试计划需要尊重“5W”原则：

“What（做什么）”：明确测试的范围和内容

“Why（为什么做）”：理解测试的目的

“When（何时做）”：确定测试的开始和结束日期

“Where（在哪里）”：给出测试文档和软件的存放位置

“How（如何做）”：指出测试的方法和工具

1.2 测试管理

● 测试管理具体包括以下内容

1. 测试方案管理：

包括单元测试、集成测试和产品测试的测试计划的录入、修改、删除、查询和打印。

2. 测试用例管理

包括测试用例的增、删、改、复制和查询；测试用例测试情况的管理，如测试状态包括未测试、测试中、已测试；测试结果分为通过、未实现、存在问题等；测试用例输入、编号和归档。

3. 测试流程管理

包括测试进度管理、测试流程标识、测试日志及状态报告。

4. 问题报告管理

问题报告处理流程（问题报告→整改报告）、实现问题报告与测试用例的关联。

5. 测试报告管理

包括生成单元测试、集成测试和产品测试的测试报告。

1.2 测试管理

● 软件测试管理系统可以对各过程进行全面控制，具体的实现过程如下

- (1) 按照国际质量管理标准，建立适合本公司的软件测试管理体系，以提高公司开发的软件质量，并降低软件开发及维护成本。
- (2) 建立、监测和分析软件测试过程，以有效地控制、管理和改进软件测试过程，监测软件质量，从而确定交付或发布软件的时间。
- (3) 制订合理的软件测试管理计划，设计有效的测试用例集，以尽可能发现软件缺陷，并组织、管理和应用庞大的测试用例集。
- (4) 在软件测试管理过程中，管理者、程序员、测试员（含有关客户人员）协同工作，及时解决发现软件问题。
- (5) 对软件测试中发现的大量软件缺陷进行合理的分类以分清轻重缓急，同时进行原因分析，并做好相应的记录、跟踪和管理工作。
- (6) 建立一套完整的文档资料管理体系。因为软件测试管理很大程度上是通过文档资料的管理来实现的。软件测试每个阶段的文档资料是以后阶段的基础，又是对前面阶段的复审。

1.3 企业的测试策略

● 企业的测试策略体现在以下几个方面

(1) 合理地减少测试工作量

通过以下方式减少测试工作量。

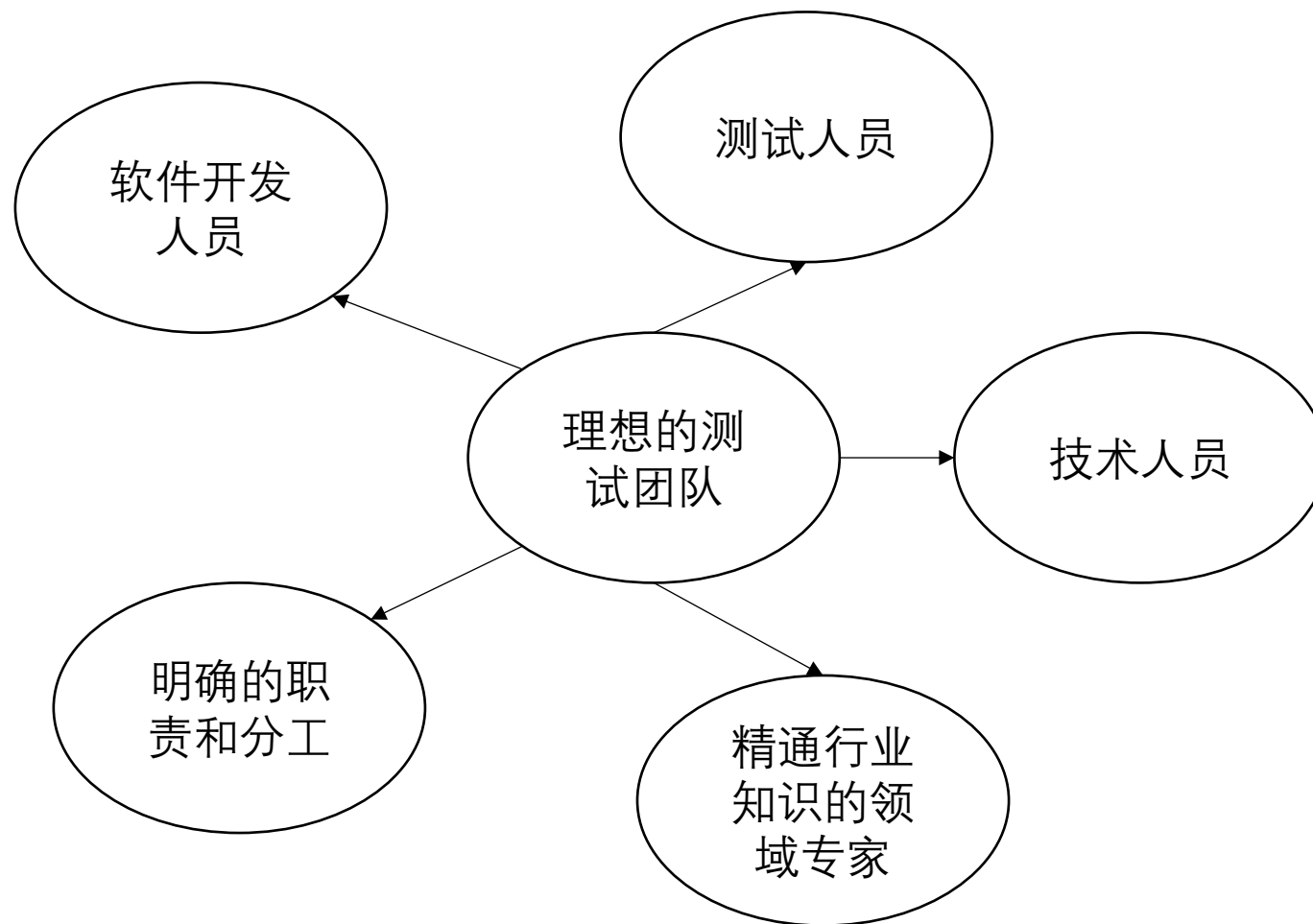
① 减少冗余的测试。在很多地方，白盒测试与黑盒测试的方式虽然不同，但往往会**产生相同的效果**。甚至在一些地方，白盒测试和黑盒测试可能会有一模一样的效果，这样的测试就是**冗余**的。

② 减少无价值的测试。无价值的测试通常是由于不懂测试技术引起的。例如，在功能测试时，本来在等价区间内**只要测试一个**典型的输入就行了，如果有人在这个区间内进行了**大量的测试**，那么除去第1次测试之外的所有测试都是无价值的。

(2) 提高测试效率

有一些项目要求“短、平、快”，且得到的经费也很少，用户对产品的质量要求也不高。为了能获取更多的利润，开发方就不得不采取一些**“偷工减料”**的方式来降低测试的代价，从而提高测试的效率。

1.4 测试人员组织



1.5 测试小组的职责

测试小组的职责

发现产品问题、评估问题严重性，并协助管理部门做出处理问题的决定。

全面质量管理

核心：提高人的素质，增强质量意识，调动人的积极性，人人做好本职工作，通过抓好工作质量保证和提高产品质量或服务质量。

基本要求：“三全一多样”，即全员的质量管理、全过程的质量管理、全方位的质量管理和多样方法的质量管理

1.5 测试小组的职责

基本的测试小组类型

质量控制组：一种权限很大的测试小组，任何技术方案和技术实施都需要接受质量控制组的检查和审核，项目实施所涉及的相关技术标准、协议和规范规则的执行情况也都需要经过质量控制组的审核。

质量保证组：协助研发对项目周期中产生的相关文档进行评审，展开内测工作，保证部门整体管理及产品质量品质的提升。

测试服务组：找出故障代码，仔细对它们进行描述，确保每个相关人员都能理解故障；该小组还要对故障进行分析，对程序质量进行估计，协助管理部门对问题的处理做出决定。

开发服务组：扩展了测试服务组的职能。

目 录

CONTENTS

01

企业测试实践

02

CMMI 和软件测试

03

DevOps

04

小结

2.1 CMMI简介

CMMI简介:

能力成熟度模型集成（Capability Maturity Model Integration, CMMI），也称为软件能力成熟度集成模型，是美国国防部的一个设想，于1994年由美国国防部与卡内基-梅隆大学下的软件工程研究中心，以及美国国防工业协会共同开发和研制的。

CMMI目的:

帮助软件企业对软件工程过程进行管理和改进，增强开发与改进能力，从而能按时地、不超预算地开发出高质量的软件。

2.1 CMMI简介

CMMI 的表述方式:

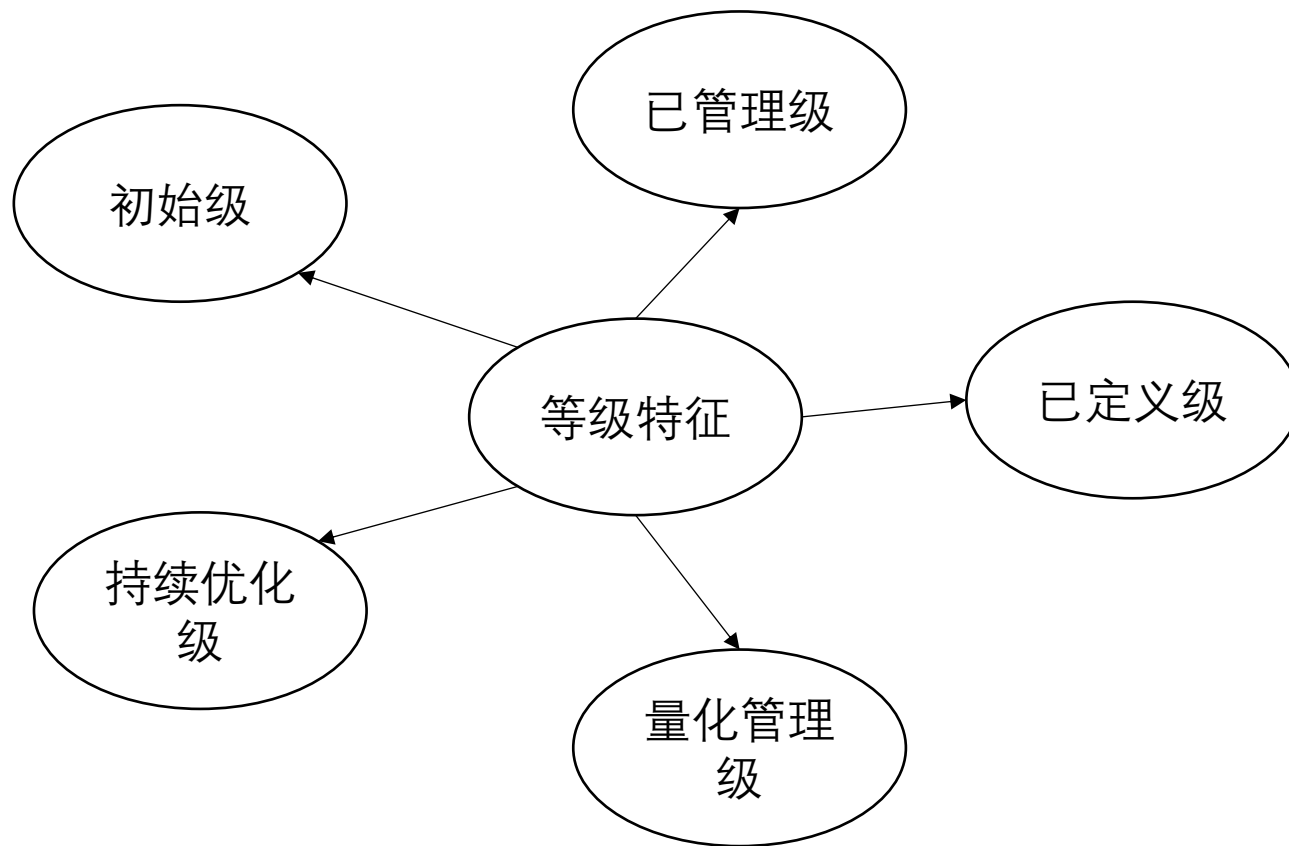
阶段式模型: 提供了预定义的路线图, 该组织级的改善是基于过程的已证实的分组和次序, 以及相关的组织级关系。

--优点: 具备管理跨组织过程的能力

连续式模型: 没有与组织级成熟度相关的分散阶段, 连续式模型的实践以支持单个过程域的增长和改善的方式进行组织, 连续式模型的每个过程域分别按照各自的能力进行评定, 评定的结果以能力特征图的形式上报。

--优点: 自由和可见性

2.1 CMMI简介



过程域的分类方式：按成熟度等级分类、按过程域紧密关系分类

其中按过程域紧密分类可以分为“过程管理类”、“项目管理类”、“工程类”、“支持类”4类。

2.1 CMMI简介

CMMI与软件测试最为紧密的两个关键过程域是验证（Verify, VER）和确认（Validation, VAL）。

● “验证”

目的在于保证工作产品满足其规定要求，该过程域有**3个特定目标**：准备验证、执行同行评审和验证选择的工作产品，要求按照需求（包括顾客需求、产品需求和产品构件需求）对产品 and 中间产品进行验证。

● “确认”

目的在于证明产品或产品构件被置于预期的环境时能够满足其预期的用途，该过程域有两个特定目标：准备确认、确认产品或产品构件。“准备确认”特定目标有**3个特定实践**：建立确认策略、建立确认环境、建立详细确认计划；“确认产品或产品构件”特定目标有**2个特定实践**：进行确认、收集和分析确认结果。

2.2 基于CMMI的软件测试流程

测试模型：

V模型： V 模型宣称测试并不是一个事后弥补行为，而是一个同开发过程同样重要的过程。该模型体现了软件开发过程和软件测试流程的关系，软件测试流程是软件开发过程的一个组成部分。

X模型： 是对V模型的改进，关于X模型的介绍详见第二章。

基于CMMI的测试流程：

从全局的角度来看，主要是在V模型的基础上在需求分析、概要设计和详细设计阶段增加了需求测试、概要设计测试和详细设计测试3个环节

目 录

CONTENTS

01

企业测试实践

02

CMMI 和软件测试

03

DevOps

04

小结

3.1 DevOps 简介

DevOps是开发（Development）和运维（Operations）的组合词，它是一套实践方法，其中包含一系列基本原则和实践。

DevOps在保证高质量的前提下缩短系统变更从提交到部署至生产环境的时间。

DevOps通过自动化“软件交付”和“架构变更”的流程，使构建、测试、发布软件能够更加快捷、频繁和可靠。

3.2 DevOps 实践

● DevOps的基本思想是“快速交付价值，灵活响应变化”

- 1、DevOps强调整个系统的稳定性，而非将性能局限于特定的工作领域里，这个领域可以大到一个部门（如开发部）或者小到一个贡献者（如开发者），由IT推动业务价值流。
- 2、DevOps通过高部署率快速地把一个想法变成价值交付到客户的手中，并且在不影响客户的基础上，快速地完成并部署很多小的变更。
- 3、DevOps的目标不仅是增加变更的频率，而且支持在不中断和破坏当前服务的基础上确保功能部署成功，同时也可以快速检测和修复软件缺陷。

3.2 DevOps 实践

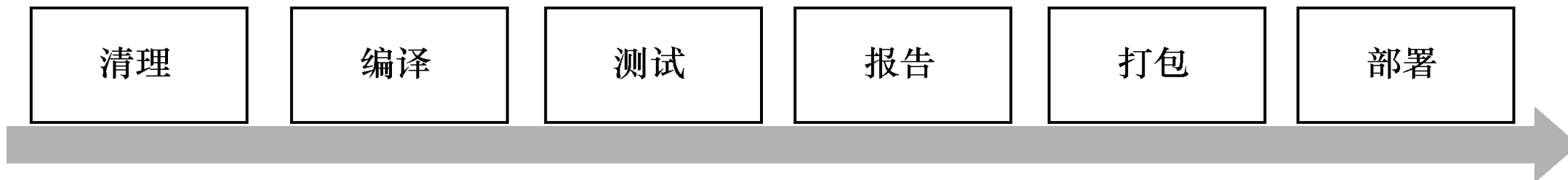
● DevOps 实践举例

(1) Jenkins使用方法如下

- ① 环境准备：下载并安装JDK，最好是JDK 1.5以上版本。
- ② 下载安装包jenkins.war。
- ③ 在安装包根路径下，运行命令“java -jar jenkins.war --httpPort=8080”。
- ④ 打开浏览器，进入链接 <http://localhost:8080>。

(2) Maven

能够自动化构建过程，从清理、编译、测试到生成报告，再到打包和部署。
Maven构建过程的部分阶段如图所示



3.2 DevOps 实践

● DevOps实践举例

(3) Docker+Jenkins打造自动化测试

Docker是一个开源的应用容器引擎，Docker可以让开发者打包他们的应用以及依赖包到一个轻量级、可移植的容器中，然后发布到任何流行的Linux机器上，也可以实现虚拟化。在该框架下，开发和测试者只要提供相应的镜像和程序就可以按照模板执行自动化任务。

- **触发条件：**由Jenkins控制，如设置定时执行，或者GitHub中的每个提交，或者每个PR执行一次。
- **构建流程：**各个环节、Job之间用Pipeline来实现整个构建流程的控制。
- **构建 Job：**一种情况是把拉取代码的过程和构建编译环境的过程都放到Dockfile里面，这适用于比较复杂的编译环境

另一种情况，为求省事方便，如Java工程，编译一般用Maven等构建工具来完成，可以放在外面，只是把运行环境搬到Docker容器里。Docker+Jenkins构建的示例代码如下图所示。

3.2 DevOps 实践

Docker+Jenkins构建

```
#!/bin/bash
#引入编译环境
export JAVA_8_HOME=$BASE_PATH/env-tools/jdk/jdk1.8.0_91
PATH=$ {GRADLE_HOME}/bin{GRAILS_HOME}/bin{NODE_HOME}/bin{GROOVY_HOME}/bin{M2_HOME}/bin{JAVA_HOME}/bin{ANT_HOME}/bin{PATH}
export PATH
export JAVA_HOME=$JAVA_8_HOME

#拉取代码，转到dev分支，编译，测试，打包
cd /root/tt
rm-rf *
git clone ssh://git@g.hz.netease.com:123/123/cloud-data-show.git
cd cloud-data-show/
git checkout --track origin/develop
mvn package

#停止删除之前生成的容器和镜像
docker kill $(docker ps -aq)
docker rm $(docker ps -aq)

#这一步删除镜像的时候，你也可以使用docker images | awk '{print $3}' | xargs docker rmi
#删除全部镜像
docker xrm test :t1

#构建镜像
set -x &&(cd /root/tt/cloud-data-show/ &&docker build -t test:t1)
#运行容器，如果是web应用，注意端口的映射，宿主机的端口不要冲突
docker run -id -p 16300:16300 --name t1 test:t1
```

目 录

CONTENTS

01

企业测试实践

02

CMMI 和软件测试

03

DevOps

04

小结

小结

这个知识点主要学习内容：

□ 企业测试实践

- 测试计划、测试管理、企业的测试策略、人员组织、测试职责

□ CMMI和软件测试

- 最为紧密的两个关键过程域是验证（Verify, VER）和确认（Validation, VAL）

□ DevOps

- 简介
- 实践：Jenkins、maven、Docker+Jenkins构建